Handoff for Claude: Artemis (Cutline Industries LLC), 2026-10-27

Prepared 2026-10-27, about 1:20 PM ET, for a fresh Claude session (claude.ai or Claude Code) that has never seen this project. All times are US Eastern (ET) unless marked UTC.

---

0. You are picking up this project

You are picking up the Artemis model work for **Latoya (Lamont) Pittman**, Cutline Industries LLC. Call her **Latoya**. Everything you need is in this file. Assume you have:

- **No access** to the assistant that did this work before you, or to its memory, tools or computer. Paths like `/workspace/...` below are on that old computer, and you can't open them. They're listed only so Latoya can find the files if she needs them.
- **Possibly access to only one GitHub repo.** Your earlier Claude Code session was scoped to `lpittman-blip/Cutline-Industries` only. The sin(X) work in sections 2–6 lives in a **different** repo, `Cutline-industries/artemis-567885` (public). Its key results, file paths, config values and commit SHAs are inlined here, so you can reason about it without access. As of Oct 27 (reported by Latoya), the **Claude GitHub app (installation 175898435) is installed on the `Cutline-industries` org with access to the selected repo `artemis-567885`**. A session refresh may be needed before you can see it. If you still can't, ask Latoya to check the app's repo selection.
- **No cloud or GPU credentials.** Anything that rents a GPU or spends money is a command **Latoya runs herself** (or approves for you) after she has OK'd a price cap and a time limit. See section 11.

There are **two separate Artemis lines of work**. Don't mix them up:

	A. Artemis 567885 (sin(X) MoE + byte LMs)	**B. Artemis 2.0 MoE code LM (gateway/artemis)**
Repo	`Cutline-industries/artemis-567885` (public)	`lpittman-blip/Cutline-Industries` (public), dir `gateway/artemis/`
What it is	A self-expanding top-k sparse MoE transformer that regresses X → sin(X) on random vectors, plus byte-level LM experiments on separate branches	A 10-expert MoE code language model trained on licence-filtered `codeparrot/github-code-clean`
Last activity	Overnight GPU run Oct 27 (cycles 125–624), PR #21 merged 12:54 PM ET Oct 27	Your previous Claude session, Oct 27: clean-label routing experiment, stopped at corpus prep
Sections	2–6	7

Resume point for line B (where you left off): section 7.5. **Resume point for line A:** section 6.

---

1. Company goal and scale ambition (all costs are estimates)

- **Goal:** best-in-world efficiency, meaning the most capability per parameter and per dollar on minimal resources. Latoya wants Artemis to compete with the big labs. She has been told plainly

that frontier scale isn't reachable on the current budget.

- **Today:** the Artemis 567885 regressor is about **213,216 total / 66,384 active parameters** (hand-counted from `expand-loop` code before the Oct 27 run, per the scaling-plan page). After the Oct 27 run it has 16 layers with 16 experts per layer (one layer has 19), so it's bigger now. **The new parameter count hasn't been recounted.** Recount it from the model code. The byte-level LM in `bytelm/` has **37,888** params (measured, PR #4).
- **Staged ambition (Latoya, Oct 27):** 100B parameters first, then 1T. A 6T option was compared. Figures below come from the planning page "Scaling Artemis 567885 toward 1 trillion parameters" (Oct 27). **Every number is an estimate.** They use 6·N_active·D FLOPs, H100 at 989 TFLOP/s BF16 and 30% utilisation, and $2.40 / $5.54 / $12.29 per H100-hour (low / mid / high, October 2026 price sources). Pilots, restarts and evals add 20–50% on top. Storage and network aren't included.

Stage	Total / active	Tokens (min, 20×active)	GPUs	Time	Compute cost (est.)
Today	213K / 66K	toy data	1 rented	hours	$1–$7 per day
1B dense	1B / 1B	20B	8 H100	14 h	$270 – $1.4K
10B MoE	10B / 2B	40B	32 H100	14 h	$1.1K – $5.5K
100B MoE	100B / 8B	160B (3T over-trained)	64 H100	4.6 days	$17K – $87K (3T tokens: $320K – $1.6M)
1T MoE (Kimi-K2-like)	1T / 32B	640B (15T frontier-style)	512 H100	9 days	$273K – $1.4M (frontier-style $6.4M – $33M)
6T MoE (untested extrapolation)	6T / 180B	3.6T (20T frontier-style)	4,096 H100	n/a	$8.6M – $44M (frontier-style $48M – $246M)

- **The planning page's bottom line:** the current pilot budget is **$600**, about 250 H100-hours at $2.40. A $7 cap buys about 2.9 H100-hours, enough for a compute-optimal model of about 160M params on about 3B tokens. That's the realistic next step for daily runs. A 1T model needs a tokenized LM on a cluster stack (Megatron-LM, DeepSpeed or TorchTitan). The sin(X) expand loop can't grow into it. The cheaper path to 1T-class capability is fine-tuning existing open weights (Kimi K2 1.04T/32B, DeepSeek-V3 671B/37B), after a licence check.
- **Data plan (estimates):** 100B needs 3.0T tokens and 1T needs 15.0T tokens. Planned mixes: DCLM-baseline 0.9T / 3.0T, FineWeb general – / 5.5T, RedPajama-v2 filtered – / 1.2T, The Stack v2 code 0.45T / 1.8T, Dolma curated 0.2T / 0.6T (plus FineWeb-Edu, FineMath, etc.). After pretraining, SFT on Tulu 3 (commercial-safe subsets), OpenThoughts3 and OpenHermes 2.5 (once its terms are cleared). Storage estimates on Azure Blob (LRS East US, per month): 100B data 20 TB is $416 hot / $304 cool / $72 cold. 1T data 82 TB is $1,706 / $1,246 / $295. 1T checkpoints (3 kept, 48 TB) are $998 / $730 / $173. The target storage account is `cutlineboxarchive` in Latoya's pilot subscription (52056bc2…). Its region and tier weren't confirmed.
- **Credits (est.):** Microsoft for Startups up to $150K (milestone-gated), AWS Activate $5K / $200K, Google for Startups AI $350K (needs VC backing). That's about $0.7M at most, and it can't be pooled into one cluster.

---

2. Repo map

2.1 `Cutline-industries/artemis-567885` (public; formerly `lpittman-blip/artemis-567885`)

The default working branch is **`expand-loop`**. `main` is only the original baseline plus docs. Stage branches are stacked PRs, and **none are merged except #4, #19 and #21.**

Branch	PR	State	Holds
`main`	—	baseline `ffc29a7` + docs (#19)	Latoya's original `README.md`, `moe.py`, `transformer.py`, `sandbox.py`
`stage-1-tests-fixes`	#1 → main	open	review fixes behind `legacy` flag, 18 tests
`stage-2-expand-engine`	#2 → #1	open	`model.py`, `expand.py`, `supervisor.py`, `physics_simulation.py`
`stage-physics-frequency-study`	#3 → #2	open	`pilot/` SIREN frequency study
`stage-byte-corpus-benchmark`	#4 → #3	**merged into #3 branch**	`bytelm/` corpus benchmark (37,888-param LM)
`bytelm-corpus-results`	#5 → main	open	seed 42/1/2 results
`repo-checks`	#6 → #3	open	five pre-integration checks, `checks/`
`stage-3-positional`	#7 → main (stacked on #6)	open	`positional.py`
`stage-4-dispatch-bench`	#8 → #7	open	`bench.py`
`stage-5-artemis-model`	#9	open	`ArtemisModel` byte LM
`stage-6-byte-training`	#10	open	`train_bytes.py`
`stage-7-alpha-cf-sweep`	#11	open	causal-safe capacity, sweep
`stage-8-routing`	#12	open	Switch top-1, z-loss, expert-choice
`stage-3-causal-positions-lm`	#13 → #2	open, **written by an earlier Claude Code session**	causal byte MoE, position ablation protocol v2
`stage-8b-physics`	#14	open	physics follow-ups
`stage-9a-fourier`	#15	open	`fourier.py`
`stage-9b-webstream`	#16	open	`webstream.py`
`fix-check3-f32-param-grads`	#17 → repo-checks	open	removes strict xfail (check 3)
`fix-check3-f32-param-grads-on-9b`	#18 → #16	open	same fix ported
`stage-9c-byte-clf`	#20 → #18	open	`clf.py` byte-level CLF
`public-polish`	#19 → main	merged	public README, CONTRIBUTING
`expand-loop-upgrades`	#21 → expand-loop	**merged 2026-10-27 12:54 PM ET (merge commit `7be7914`)**	Oct 27 upgrades + session results
`expand-loop`	—	head `7be7914`	the live expand loop, cycles 0–114 history in `config.json`, session results
`dense-model`	—	no PR	`artemis_dense/` dense-transformer rebuild (section 4.4)
`license-draft`	—	—	Apache-2.0 draft, not chosen yet
`archive/*`	—	—	archived reference code

Key files on `expand-loop` (`7be7914`):

- `moe.py`: `CapacitySparseMoELayer` (top-k router, aux balance loss, capacity drops). `legacy=True` reproduces the original numbers.
- `transformer.py`: attention + MoE block. `model.py`: `DynamicMultiBlockTransformer` (`add_layer` = exact identity init, `add_expert` = copy of the most-loaded expert + noise).
- `expand.py` (438 lines): the X → sin(X) task and growth rules. CLI: `python expand.py init|run-cycle|loop [--cycles N] [--root DIR] [--force]`. Rules: dropped>0 → capacity_factor += 0.05. aux>1.2 → alpha ×1.5. Loss stall (<1% rel. improvement) → add an expert to the deepest layer still below 16, or add a layer.
- `supervisor.py`: runs cycles in subprocesses with backup, `ast` check, timeout, shape auto-correct and rollback. CLI: `python supervisor.py --cycles 10 --timeout 900 [--max-attempts 3]`.
- `evaluate.py` (new Oct 27): fixed 1,024-sequence held-out set with its own seed. Reports capped and dropless MSE, R^2, |x| buckets, OOD tails |x|∈[2,3.5], odd-symmetry error, token-order interference, zero/identity baselines, and the legacy 32-seq eval loss. CLI: `python evaluate.py --root . --device cpu --out eval.json [--ckpt PATH] [--n-seqs 1024]`.
- `config.json`: the single source of truth (architecture, rules, caps, full per-cycle `history`). **Caps now: `capacity_factor_max` 3.0, `num_experts_max` 16 (per layer), `alpha_max` 1.0, `num_layers_max` 16.** The `config.json` committed on `expand-loop` is still the **cycle-115 state** (12 layers, experts `[6,9,5,8,5,5,5,19,7,9,5,10]`, CF 2.5). The cycle-624 state is in `runs/session-20261027/vast-main/final_config.json` (2.7 MB, includes history).
- Tests: `tests/` (test_moe, test_model, test_expand, test_supervisor, test_physics, test_upgrades) and `checks/` (check1–5). Run with `python -m pytest -q tests checks -p no:cacheprovider`. **`checks/` needs `pyjwt` installed.** Collection fails with `No module named 'jwt'` otherwise (seen on Oct 26 and 27). The import comes from `pilot/artemis_physics_*.py`. There's no `requirements.txt`.
- Session scripts: `runs/session-20261027/ablation-vast-seed0/session_vast_ablation.sh` (ablation), `runs/session-20261027/session_main.sh` (main run; its header comment says Azure ML, but it's the script the Vast main run used), `runs/session-20261027/ablate_aml.sh` + `job.yml` (Azure ML replicate).
- Docs: `docs/REVIEW.md` (8 verified bugs in the original), `docs/ARTEMIS_567885_RESULTS_FOR_CODEX.md` (early results handoff), `PLAN.md`, `PROGRESS.md`, `docs/INTERNET_COORDINATE_FIELD.md`, `docs/CONTINUOUS_LANGUAGE_FIELD.md` (truncated user spec).
- Releases (both **draft**): `ckpt-expand-loop-upgrades-20261027` holds `checkpoint_cycle155.pt` … `checkpoint_cycle625.pt` and `final_checkpoint.pt` (about 11 MB each). `checkpoint_cycleN.pt` is the state *before* cycle N, so `checkpoint_cycle625.pt` = `final_checkpoint.pt` = after cycle 624. `artemis-100m-step5250-preview` holds the bf16 weights of the separate 153M code LM (section 4.6).

How to run on a GPU host (Latoya runs this, never on the old assistant's box):

```
git clone https://github.com/Cutline-industries/artemis-567885 && cd artemis-567885 && git checkout expand-loop
pip install pytest==8.3.3 pyjwt==2.9.0        # image used: pytorch/pytorch:2.5.1-cuda12.4-cudnn9-runtime
python -m pytest -q tests checks -p no:cacheprovider
gh release download ckpt-expand-loop-upgrades-20261027 -p final_checkpoint.pt -R Cutline-industries/artemis-567885
mkdir -p runs && cp final_checkpoint.pt runs/checkpoint.pt
cp runs/session-20261027/vast-main/final_config.json config.json   # cycle-625 state, recipe C keys
python evaluate.py --root . --device cpu --out eval_now.json
python supervisor.py --cycles 10 --timeout 900                      # continues from cycle 625
```

2.2 `lpittman-blip/Cutline-Industries` (public): Artemis 2.0 code LM

`main` is at `50fc779` (2026-10-26). The relevant code is under `gateway/artemis/`:

- `model.py:26`: `EXPERT_NAMES = ("intent","architecture","planning","build","test","deploy","memory","security","observability","cost")`. `model.py:44,47`: `n_experts`/`expert_names` config fields. `model.py:191-196`: the supervised router loss (NLL on tokens with label ≥ 0). `model.py:260`: labels withheld when `route_coef` is 0. `model.py:281-283`: `loss += route_coef * route`.
- `data.py:19` / `stream_prepare.py:45`: `UNLABELED = 255`. `data.py:53`: `label = EXPERT_NAMES.index(domain) if domain else UNLABELED`.
- `code_corpus.py:151` `classify(..., residual_build=True)`, `:177` the catch-all `build` label, `:186` `build(..., residual_build=True)`, `:225` `rec = {"text": code}`, `:242-246` `--no-residual-build` CLI.
- `stream_corpus.py` (243 lines): download → filter → tokenise → delete, one shard at a time, resumable through `prep-state.json`. `stream_prepare.py:167` hardcodes `"domain_labels": True` in the manifest.
- `scripts/train/stages.sh` (`ROUTE_COEF` defaults to 0 at line 57), `proto-routing-ablation.sh`, `proto-label-ablation.sh`, plus cloud wrappers under `scripts/train/{kaggle,azure,ibm}/`.
- Docs: `docs/MOE-ROUTING-ABLATION.md`, `docs/PROTO-SPECIALISATION-117M.md`, `docs/proto-moe-vs-dense.md`, `docs/PRETRAIN-BUDGET-80B.md`, `docs/ARCHITECTURE-AUDIT-14B.md`, `docs/CODEX-HANDOFF-ARTEMIS.md`, `docs/TRAINING-HANDOFF.md`.

2.3 Other Artemis-related repos (all private unless noted; low priority)

`lpittman-blip/artemis-ai` (the same `artemis/` package lineage as gateway/artemis), `artemis2-finetune` (Qwen LoRA/QLoRA prep; no GPUs were created from it), `artemis-567885-rebuild` (a copy), `artemis-gpu-run-kit`, `artemis-gpu-run-kit2`, `artemis-qwen-kit`, `artemis-glimmer-kit`, `artemis-llama-conv-kit` (GPU rental and fine-tune scripts; no weights or keys), `artemis-pb-infra` (Azure petabyte data-plan infra code, unfinished), `artemis-cli`, `artemis-dashboard-site` / `artemis-dashboard-collector` (stats dashboard), `portfolio` (public), `lpittman-blip.github.io` (public site). In the org: `Cutline-industries/.github` (org profile).

---

3. Results ledger: Artemis 567885 (sin(X) MoE), newest first

The model is d_model 16, 2 heads, d_ff 64, top-2. The task is X → sin(X) on random vectors, seq_len 16, steps_per_cycle 300, lr 1e-3, seed 567885. "Loss" is MSE. The original 32-seq eval loss is the per-cycle `loss` in the logs. "Held-out" is `evaluate.py`'s 1,024-seq set (Oct 27 onward).

3.1 2026-10-27 overnight session (cap $20 total, 6 h from 02:28 ET, teardown after)

The code changes are in commits `3b3e8b5` (moe.add_expert device fix), `79cc7ff` (device option, cosine+warmup, Adam carry-over on growth, EMA, dropless eval, checkpoint loaded once, CPU-portable checkpoints), `19392f7` (evaluate.py + tests), `2bd14d2` (caps 16/3.0), `1dc2b53` (caps test), `a54d15b` (`data_seed`). Everything is opt-in through config keys. Results commits: `3a44a05`, `00260bb`, `8d3d834`, `cbec811`, `51d8967`, `fd0efe6`, `8c95af3`. PR #21, final comment: https://github.com/Cutline-industries/artemis-567885/pull/21#issuecomment-6441851862

(a) Seed-0 recipe ablation: Vast.ai instance 57927018. RTX 3060 Laptop GPU (12 GB) + Ryzen 9 7950X (32 threads), $0.097/h. torch 2.5.1+cu124, CUDA 12.4, Python 3.11.10, numpy 2.1.2, pytest

8.3.3. Ran 06:30–06:35 UTC (02:30–02:35 ET). Destroyed at 02:37 ET.

- Start: the cycle-114 checkpoint (`313e6fa`). Each arm ran 10 supervisor cycles (115–124), `--cycles 10 --timeout 900`. Common settings: `ema_decay 0.99`, `eval_extra true`, `num_threads 4`.
- Arms: **A** = original recipe (caps raised only). **B** = A + `lr_schedule cosine`, `warmup_steps 30`, `lr_min 1e-5`, `carry_optimizer_on_growth true`. **C** = B + `batch_size 64`. **G** = C on `device cuda` (timing only).
- Before (cycle 114, raw): held-out 0.023703, dropless 0.023642, R^2 0.945261, OOD 1.343249, odd-sym 0.003591.

Arm	layers @124	held-out raw	dropless raw	EMA	EMA dropless	R^2 raw	OOD raw	odd-sym raw	wall s
A	14	0.028474	0.023560	0.022899	0.022761	0.934241	1.346141	0.013087	215
B	14	0.022839	0.022838	0.022741	0.022741	0.947257	1.335597	0.001937	207
C	15	**0.022304**	0.022291	**0.022260**	0.022260	0.948492	1.366773	0.001218	289
G (C on CUDA)	15	0.022309	0.022297	0.022408	0.022271	0.948480	1.376723	0.001231	277

- `winner.json` picked **C** (best of min(raw, EMA)). The script then keeps whichever of C/G ran faster, so `status.txt` logs `winner=G`. Same recipe either way.
- Tests: the first pytest run failed collection (rc=2, missing `jwt`). The second run gave **147 passed, 1 failed, 1 xfailed**: `tests/test_expand.py::test_default_caps_are_the_approved_limits` still expected caps 2.5/12. Fixed in `1dc2b53`. The third run gave **148 passed, 1 xfailed**. The xfail is the known check-3 fp32 weight-grad tolerance (`checks/test_check3_moe_dispatch.py::test_bench_size_float32_param_grads_at_stated_tol`), fixed by unmerged PRs #17/#18.
- Raw output: `runs/session-20261027/ablation-vast-seed0/` (arm consoles `arm_*.txt`, `eval_*.json`, `winner.json`, `pytest2.txt`, `pytest3.txt`, `env.txt`, `nvidia_smi.txt`).
- Cost: **$0.0136 real** (Vast credit 15.337039 → 15.323407).

(b) Seed-1 replicate: Azure ML job `sad_sprout_y6p1tbpjkg`. Compute `t4-a567-20261027` (Standard_NC8as_T4_v3, 1× T4 16 GB, East US, dedicated, Latoya's pilot subscription 52056bc2…). Image `pytorch/pytorch:2.5.1-cuda12.4-cudnn9-runtime`, torch 2.5.1+cu124, 8 CPUs.

- Ran 06:44 UTC until the **80-minute job timeout** killed it at 08:04 UTC (02:44–04:04 ET). That was a timeout, not a code error.
- Wave 1 (`data_seed 1`, arms A/B/C/G, cycles 115–124, `num_threads 2`) finished. Wave 2 (`data_seed 2`) never logged completion. `evaluate.py` JSONs weren't written to the mounted output, so only the per-cycle **32-seq eval loss** exists. At cycle 124: **A 0.026141 > B 0.024871 > C 0.024491** (G 0.024290). That's the same order as seed 0. Arm times: B 316 s, A 383 s, G 677 s, C 1394 s.
- Raw output: `runs/session-20261027/ablation-azure-seed1/` (`A_s1.txt`…`G_s1.txt`, `NOTE.md`, `status.txt`). Compute deleted at about 04:15 ET.
- Cost: **about $1.05, an estimate** (about 1 h 24 m node time × $0.752/h). Not billed yet at time of writing.

(c) Main run: Vast.ai instance 57928114. Same hardware class (RTX 3060 Laptop 12 GB + Ryzen 9 7950X, $0.097/h) and software (torch 2.5.1+cu124, Python 3.11.10).

- Recipe C from the arm-C cycle-124 checkpoint (15 layers): cosine lr 1e-3 → 1e-5, warmup 30, Adam carry-over on growth, batch 64, EMA 0.99, `num_threads` 16, `steps_per_cycle` 300.

- One timed cycle per device picked **cuda** (22 s vs 31 s CPU). Then 50 supervisor chunks of 10 cycles (`supervisor.py --cycles 10 --timeout 900`) ran **cycles 125–624**, 06:44–11:58 UTC (02:44–07:58 ET), with **0 failed chunks**. Chunk time grew from 215 s to about 420 s. Held-out eval on CPU after every chunk.
- Script: `runs/session-20261027/session_main.sh`. Raw output: `runs/session-20261027/vast-main/` (`status.txt`, `curve.jsonl`, `eval_start.json`, `eval_after.json`, `final_config.json`, `pytest.txt`, `env.txt`, `timing_*.txt`). Checkpoints are on the draft release `ckpt-expand-loop-upgrades-20261027`.
- Tests on the host: **148 passed, 1 xfailed** in 38.07 s.

Metric (1,024-seq held-out, scored on CPU)	Start (cycle 124)	End (cycle 624)
Held-out MSE, capped, raw	0.022304	0.025808 (worse)
Held-out MSE, capped, EMA	0.022260	0.023643 (worse)
Held-out MSE, dropless, raw	0.022291	**0.021700** (−2.6%)
Held-out MSE, dropless, EMA	0.022260	**0.021689**
R^2, raw / EMA	0.948492 / 0.948593	0.940399 / 0.945400
OOD MSE (|x|∈[2,3.5]), raw / EMA	1.366773 / 1.364052	1.990560 / 1.954657 (worse)
Odd-symmetry error, raw / EMA	0.001218 / 0.001127	0.013233 / 0.006075 (worse)
Bucket MSE |x|<1, raw	0.003746	0.008709
Token-order error	0.0	0.0
Legacy 32-seq eval loss, raw	0.024371	0.028526
Baselines (zero / identity)	0.433015 / 0.217508	same
Architecture	15 layers, [6,9,5,8,5,5,5,19,7,9,5,10,5,6,4]	16 layers, all 16 experts except layer 8 at 19

- Capped curve (`curve.jsonl`, raw held-out): it hovers at 0.0220–0.0229 through cycle 284, then jumps to 0.024111 at cycle 294. After that it oscillates between 0.0232 and 0.0361 (worst 0.036136 at cycle 524). OOD rises steadily from 1.38 to 1.99.
- **Reading:** dropless held-out kept improving. The capped score regressed from about cycle 284, when every layer filled to 16 experts and eval-time capacity drops climbed. The extra experts added drops faster than skill, so **training longer won't fix it**.
- Teardown: Vast destroyed and the account showed 0 instances. On-instance self-destroy at 08:25 ET plus box-side killers were in place.
- Cost: **Vast $0.59 real for the whole session** (credit 15.337039 → about 14.7457, both instances). Azure about $1.05 estimated. **Total about $1.64 of the $20 cap.**

3.2 2026-10-26 daily GPU session (cap $1)

Vast.ai **57817513**, 1× RTX 4070, offer 45140324, image `pytorch/pytorch:2.5.1-cuda12.4-cudnn9-runtime`, $0.167/h. Rented 09:08 ET, destroyed 09:19 ET (verified). **$0.0336 real** (credit 15.3707 → 15.3370). The 9c result was earlier reported in PR #20 as "$0.034 measured". Same session.

- **Expand loop cycles 105–114** (`supervisor.py --cycles 10 --timeout 600`), commit `313e6fa`: no rollbacks. Layer 11 grew 5 → 10 experts on stalls. CF stayed at cap 2.5. Final eval loss **0.02585**, best **0.025364** at cycle 108. aux 1.001021, 12 layers, 93 experts total.
- **Tests:** `expand-loop` tests+checks: 1st run collection error (missing `jwt`), 2nd **141 passed, 1 xfailed** (52.2 s). `sandbox.py`: fixed 0.3415 / legacy 1.5083.
- **stage-9c branch** tests+checks: **401 passed, 6 failed**. Five are bit-exact causality tests that fail multi-threaded on that CPU and pass with `OMP_NUM_THREADS=1` (10/10 in the 1-thread rerun):

`test_artemis_model.py::test_causal_future_bytes_do_not_affect_past[loop-none|loop-sinusoidal|loop-learned]`, `test_positional.py::test_causal_future_change_does_not_affect_past[sinusoidal]`, `test_stage7_capacity.py::test_sequence_independent_of_other_sequences`. One (`test_clf.py::test_cuda_matches_cpu`, NameError `ByteCLF`) was a test bug, fixed, after which clf passed 26/26.

- **dense-model branch** (`a567-dense`) tests: **207 passed, 4 failed, 5 skipped**. All 4 failures are `RuntimeError: use_bwrap=True but bubblewrap is not usable on this host` (container limitation, not a model bug): `test_dense_engines.py::test_iterate_fixes_failing_snippet`, `test_new_engines.py::test_math_runs_in_sandbox`, `test_sandbox_promotion.py::test_sandboxed_run_cannot_read_workspace_or_reach_network`, `test_sandbox_promotion.py::test_watch_practice_promotion`. dense checks: **87 passed, 1 xfailed**.
- **Stage 9(c) ByteCLF on CUDA** (`clf.py --device cuda --dispatch vectorized --variants doc,none,window --steps 3000 --eval-every 250`), 732,800 params, one seed, one book (Pride and Prejudice, last 10% held out, 74,624 bytes). Held-out bits/byte: unigram 4.6192, none 2.0745, doc 1.7641 (train windows 1.4452), **window 1.6621**. Position helps. The absolute document coordinate overfits most. PR #20.
- Oct 27 daily session: **skipped** (the overnight run covered it), $0.

3.3 Expand-loop history on the box CPU, cycles 0–104 (no cloud charges)

From the `config.json` history and commit messages on `expand-loop`. These are 32-seq eval losses.

Cycles	Commit	Date (ET)	End loss	CF	Layers	Experts total	Notes
0–9	`7134062`	Oct 23 00:17	0.0965 → 0.052061	0.8 → 1.2	1 → 2	4 → 10	first real run, no rollbacks
10–19	`8420c60`	Oct 24 08:58	0.03215	1.25	2	12	
20–29	`59df9f4`	Oct 24 11:58	0.049488	1.45	3	19	
30–39	`2feabe3`	Oct 24 14:58	0.027932 (best 0.0273 @38)	1.70	4	26	
40–49	`9175733`	Oct 24 17:55	0.02641	1.75	6	37	
50–59	`80ffa76`	Oct 24 20:54	0.026348	1.85	8	49	layer cap 8 reached
60–69	`a437941`	Oct 25 08:52	0.025595	2.0	8	55	CF cap 2.0 at cycle 66
70–79	`de37c7a`	Oct 25 11:59	0.028093	2.0	8	62	last layer 12 → 19 experts
(caps)	`80ef9bb`	Oct 25 14:00	—	—	—	—	caps raised to CF 2.5 / 12 layers; 16 experts becomes a per-layer cap
80–89	`54e5264`	Oct 25 15:08	0.025935	2.45	10	74	
90–99	`afa13f5`	Oct 25 18:14	0.02638	2.5	10	78	CF cap 2.5 at cycle 90

Cycles	Commit	Date (ET)	End loss	CF	Layers	Experts total	Notes
100–104	`3bc5e10`	Oct 25 21:18	about 0.026	2.5	12	—	**cap hit**: 12 layers. Cycle 100 timed out twice under box CPU contention, rolled back, then passed with `OMP_NUM_THREADS=1`, timeout 600 s
105–114	`313e6fa`	Oct 26 09:15 (GPU)	0.02585	2.5	12	93	see 3.2

Latoya first chose to keep the caps at 12 layers / CF 2.5 (Oct 25–26), then approved 16 / 3.0 on Oct 27 (`2bd14d2`). **The merge of PR #21 makes 16 / 3.0 the default on `expand-loop`.**

3.4 Stage / correctness results on stage branches (box CPU, no cloud charges unless stated)

Environment unless noted: Python 3.13.5, torch 2.14.1+cpu, pytest 9.1.1, 8 CPU threads (Intel Xeon).

- **Baseline (Oct 23, `ffc29a7`):** `python sandbox.py` gives Total Loss 1.5083, dropped 6, ∇x 0.1581, ∇W_g 0.0546. Every branch keeps `--mode legacy` reproducing this exactly. Fixed mode: 0.3415.
- **Stage 1 (#1):** 8 verified findings in `docs/REVIEW.md` (capacity cut by token order not priority, dropped-token double-count, noisy/top-1-only aux, `sum==0` drop heuristic, no causal mask, no positions, dead top-1 gate gradient (0.000000), drop count > tokens). **18 tests pass.**
- **Stage 2 (#2):** 47 tests pass. Physics PINN 120 s / 6,358 steps: PDE residual 3.40e-4, IC 1.53e-4, BC 1.18e-5, rel L2 0.055 / 0.157 / 0.250 at t = 0 / 0.5 / 1. Router collapse: top-1 counts [0, 0, 1129, 8132].
- **Physics pilot (Latoya's spec, Oct 23):** did **not** converge. 1,000 epochs gave test MSE 0.3567, rel L2 0.852. Normalised inputs + 5,000 epochs gave 0.0935 / 0.436.
- **Frequency-and-capacity study (#3), 667 s CPU:** SIREN 32×2 (1,217 params) reached val MSE ≤ 1e-4 in 5/5 seeds at k = 1, 2, 4 and 0/5 at k = 8. The (x,t)-only model got 5/5 at k = 8. No width/depth config with 3 inputs got there at k = 8. Selected: SIREN w0_first 3 / w0_hidden 10, lr 3e-3. The formula was 1.3–15.9× faster than every net. Outputs: `pilot/results/`.
- **bytelm corpus benchmark (#4, #5):** 37,888-param byte LM. Causal check: future bytes leave earlier logits exactly unchanged. Ledger rerun: test BPB 4.5745 vs unigram 5.7727 (a pasted 7.018 was **not** reproduced). Seeds 42 / 1 / 2, 10 epochs × 1,000 steps, Gutenberg corpus (12 MB train): test BPB 3.5860 / 3.5535 / 3.5551 vs unigram 4.7079, about 28–30 s each.
- **Repo checks (#6):** `pytest tests checks` gave 134 passed, 1 xfailed. Check 3 fp32 weight grads at N=4096 reached max 3.05e-5 (strict xfail, fixed in #17 → 139 passed, 0 xfail). **Check 5: capacity routing leaks the future into the past (max 0.366 at CF 1.0). Dropless doesn't leak.** PINN held-out rel L2 vs FD 0.1425.
- **Stage 3 (#7):** 148 passed, 1 xfailed. **Stage 4 (#8):** vectorized dispatch is faster only at E=64, d=64 (1.15–2.97×) and slower at d=256 fwd+bwd. Default stays `loop`. 157 passed, 1 xfailed.
- **Stage 5 (#9):** `ArtemisModel` byte LM. 180 passed, 1 xfailed. Overfit check: 8.148 → 0.410 bpb.
- **Stage 6 (#10), CPU 900 s, 2026-10-24 15:01–15:16:** 5,399,296 params (d 256, 4 layers × 4 experts), 2,593 steps. Held-out val (Austen) **1.7605 bpb** vs unigram 4.5814. Test (Twain) 1.9746 vs 4.7076. 191 passed, 1 xfailed.
- **Stage 7 (#11), 20-run α×CF sweep, 2,289 s:** CF matters most (CF 0.8 costs +0.13–0.16 bpb; CF ≥ 1.25 lands within 0.01 of dropless). α barely matters. Best 1.9809 (α 0.3, CF 1.5), single seed. 223 passed, 1 xfailed.
- **Stage 8 (#12):** top-1 renorm 2.1087 vs top-2 baseline 1.9866. Top-2 + z-loss 1e-3 gave 1.9767. Expert-choice is labelled LEAKY. 274 passed, 1 xfailed.

- **Stage 3 + language benchmark (#13, written by an earlier Claude Code session):** protocol v2, 4 arms × seeds {42, 1, 2}, 5,000 steps, 37-book author-split corpus. Test bpb: dense (37,888 params) **3.037 (sd 0.056)**, moe-learned 2.826 (0.064), **moe-sinusoidal 2.730 (0.032)**, moe-fourier 2.754 (0.026). Unigram 4.545. The Fourier vs sinusoidal difference is mixed in sign, so it's not evidence. Report: `experiments/language/results_v2.md`, `experiments/language/HANDOFF.md` on that branch.
- **Stage 8b physics (#14), 46 runs, about 18.5 min:** best config lr 1e-4 + α 0.1 + noise 1.0 + curriculum 0.5 gave rel L2 at t=1 **0.0077** (4-seed mean 0.0082) vs baseline 0.249662. Noisy routing drives the gain. 295 passed, 1 xfailed.
- **Stage 9a (#15):** `fourier.py`, 37 tests, 332 passed, 1 xfailed. **Stage 9b (#16):** `webstream.py`. Real Range fetch of `pg1342.txt` returned 206 for 4,096 B in 0.289 s, with a sha256-verified resume. 376 passed, 1 xfailed.

---

4. Results ledger: other Artemis models (LLMs and fine-tunes)

Outputs for these runs sit on the old box under `/workspace/a567-dense/checkpoints/` (gitignored, not on GitHub). Kit scripts are in the private `artemis-*-kit` repos.

4.1 Dense rebuild, CPU (branch `dense-model`, Oct 25, no cloud charges)

- Head commit `56222e8`: `artemis_dense/train.py` (Stage 1 pretrain + Stage 2 assistant-only SFT), byte or BPE tokenizer.
- `byte_sft2`: 6,885,632 params, byte tokenizer, 150 SFT steps. Held-out 8.074 → **1.6932 bpb**. `bpe_sft`: 7,868,672 params, vocab 4096, 3.882 → **1.6912 bpb**.
- The samples are mostly dictionary-style gibberish ("anticonvulsant (noun): ..."). It's not a usable assistant.
- `Accelerate.md`: a measured 5-minute CPU A/B, baseline **2.2341** vs "fast" preset 2.2902 held-out bpb. The fast preset over-fitted.

4.2 Dense GPU run 1, Oct 25: "Artemis 567885 dense" 48.8M

- Vast.ai **57746707**, 1× RTX A6000, $0.40/h + storage. Rented 21:16, destroyed 22:14 ET. **Billed $0.442** (GPU 0.974 h $0.390 + storage $0.016 + download $0.005 + upload $0.031).
- Model: decoder-only dense, 12 layers, d_model 512, 8 heads / 2 KV heads (GQA), d_ff 1536, vocab 24,576 BPE (`tok24k`), context 1024, **48,771,584 params**. Seed 567885. torch 2.5.1+cu124.
- Data: 83.18M training tokens (filtered web 72.0M + WordNet 4.6M + Artemis docs upsampled ×40 = 6.6M, 7.9% share). Web filter: 119,178 docs in, 76,995 kept, 4,669 dups, 37,514 filtered, 6,035 PII lines stripped.
- Command: `python gpu_train.py --run medium_bpe24k --size medium --tok tok24k/tokenizer.json --context-len 1024 --dropout 0.0 --batch 32 --lr 6e-4 --warmup 500 --max-epochs 4.5 --stage1-budget-s 7200 --eval-every 500 --patience 6 --web-val-windows 200 --stage2-steps 600 --sft-batch 16 --sft-lr 1e-4`
- Stage 1: 11,422 steps, 374.3M tokens, 3,020 s. Web val bpb 3.2927 → **1.1906**. Artemis val 4.036 → **1.374**. Stage 2 SFT: best Artemis val 1.4293, final 1.5356 (web 1.2120). SFT made held-out *worse*.

4.3 Dense GPU run 2, Oct 25–26: about 94M dense on FineWeb-Edu (unverified outcome)

- Vast.ai **57753825**, rented 22:29 ET Oct 25. The harvester couldn't connect, and at 04:49 ET the instance was already "not found". **No results were harvested.** Its cost is inside the combined figure in 4.5.

4.4 Artemis-7B: QLoRA of Qwen2.5-7B-Instruct, Oct 25

- Vast.ai **57758675** (2× RTX 4090, DDP). Rented 23:19, destroyed 23:36 ET. Merge, eval and GGUF ran on 57760296 and 57761589 (1× RTX 4090). Cap $5.
- QLoRA NF4/bf16, LoRA r 32, α 64, dropout 0.05, 196 modules, **80,740,352 trainable**. lr 1.5e-4, bsz 4 × accum 2, max_len 1024, 2 epochs planned, eval every 25, patience 3, seed 567885. torch 2.8.0+cu128, transformers 5.20.2, peft 0.21.2.
- Data: 3,051 train examples (Artemis SFT 363 ×2 + databricks-dolly-15k 2,350, CC-BY-SA-3.0), 699,837 tokens.
- Early stop at step 200, best step 125. Val loss 2.713 → **1.5843**. Wall 542.6 s. Q4_K_M GGUF (4.68 GB) made.
- Eval (10 prompts, greedy): correct identity and approval behaviour. **Known flaw:** "Are you Qwen?" → claims "Artemis 567890 … trained by Anthropic" (hallucination; needs identity negatives).

4.5 Artemis-30B: QLoRA on Meta "Muse Glimmer 30B", Oct 26

- Vast.ai **57759536** (2× L40S, DDP). Rented 23:29 Oct 25, destroyed 00:16 ET Oct 26. Cap $15.
- Base `meta-models/Muse-Glimmer-30B`, Apache-2.0 (licence verified before download), 29.78B dense, multimodal (vision tower not adapted).
- LoRA r 32 / α 64, 416 modules, **209,584,128 trainable**. lr 1.5e-4, bsz 2 × accum 4. 3,052 examples, 731,686 tokens. Early stop at step 250, best step 175, **val loss 1.4213**, wall 1,783 s.
- Eval: correct identity and approval answers. Outputs carry a stray " to=user" channel header (cosmetic). Adapter only; there are no merged weights.
- **Combined real Vast spend for 4.3 + 4.4 + 4.5:** credit 19.2062 → 15.3707 = **$3.84**, assuming no top-ups. The per-run split is **unverified**.

4.6 artemis-100m (153M MoE code LM), Oct 22–24

- Kaggle notebook `lamontpittman1/artemis-100m` (free T4). Stopped at step 5,442 of 22,533, val loss about 2.51. The checkpoint at step 5,250 is on draft release `artemis-100m-step5250-preview`.
- Resuming on a Nebius H100 was approved and then **cancelled** on Oct 23. The H100 job never ran.
- The step-5,250 checkpoint went live on Oct 24 as an opt-in "Artemis preview" in the site chat (CPU, bf16) on Azure VM `Vm`. That VM is in a subscription that must not be touched (section 10).
- Its training recipe (from `box-workspace-code` README): warmup 450, cosine → 10%, seed 1337, `route_coef 0`.

4.7 Not trained / not Artemis

- `artemis-nvidia-preview` is a chat proxy to NVIDIA-hosted Nemotron models. It is **not** an Artemis model and must never be presented as one.
- The first A6000 "Artemis 2.0" fine-tune (Oct 25) was stopped at step 14 of 96, cost about $0.12, and was never rerun.

- The Llama-3.1-8B conversational fine-tune (approved Oct 26, $4 cap): **no evidence it ran.**
- The Oct 11 Nebius 8×H100 run by an earlier Claude agent was stopped and blocked by service-account permissions. Nebius stays stopped.

---

5. Current state of Artemis 567885 (as of 1:10 PM ET Oct 27)

- `expand-loop` head is `7be7914` (PR #21 merged). **Default caps are now 16 layers / CF 3.0 / 16 experts per layer.** Latoya had earlier chosen 12 / 2.5. The merge changed the default.
- **PR #21 was merged from the `lpittman-g` account. Use `lpittman-blip` only.**
- The latest weights (after cycle 624) are `final_checkpoint.pt` on draft release `ckpt-expand-loop-upgrades-20261027`. The matching config is `runs/session-20261027/vast-main/final_config.json` (cycle 625 next). The committed `config.json` and the old box's local checkout are still at **cycle 115**.
- The daily 9 AM ET session ($1/day cap, one rented GPU, teardown after) would continue from cycle 625 with recipe C. With every layer at 16 experts, every new stall now adds layers only, and layers are capped at 16. **Growth is effectively saturated.**
- Open stacked PRs #1–#3, #5–#18 and #20 are unmerged. Merge order follows the stack.

---

6. Open problems and recommended next steps (Artemis 567885)

1. **Fix the eval-time capacity drops (top priority).** Capped held-out MSE regressed from about cycle 284 while dropless improved. Options to test, each as one ablation against `final_checkpoint.pt` with the same held-out set: (a) dropless eval/inference (`capacity_factor=None`), which check 5 also shows is the only causal-safe mode; (b) a separate, larger eval-time CF; (c) slower expert addition: higher stall threshold, cap total experts, or prune low-load experts; (d) Codex's control experiment, a fixed-architecture baseline with matched steps/FLOPs, to show growth helps at all. Report raw and EMA, OOD and odd-symmetry too.
2. **Merge PRs #17/#18** (or cherry-pick `b4cf97d`) to drop the last xfail. **Add `requirements.txt`** (torch, pytest, pyjwt, numpy), or remove the `jwt` import from `pilot/`.
3. **Code cleanup found during this handoff:** `session_main.sh`'s header says Azure ML although Vast ran it. `final_config.json` (2.7 MB) is committed with the full history. Consider trimming history or storing it as JSONL. `winner.json` and `status.txt` disagree in wording (C vs G, same recipe). `config.json` on the branch is stale (cycle 115) relative to the release checkpoint. Add a script that pulls the release checkpoint and config together.
4. **Distillation is for the language models, not the sin(X) regressor.** Meta granted Latoya gated access to **Llama-3.1-8B** via Hugging Face. Her token is stored on her machine (on the old box it was `~/.config/hf_artemis_token`). Never paste its value. Use Llama-3.1-8B (licence requires a "Built with Llama" display) as the teacher for bytelm/dense or the code LM. **Qwen2.5-7B-Instruct** (Apache-2.0) is the fallback, already used for Artemis-7B. Fix the identity hallucination with identity-negative SFT examples.
5. **Scaling ladder:** the next realistic step is a compute-optimal ~160M model on ~3B tokens (about $7 at $2.40/H100-h, estimate). Only move up when measured eval gains match the scaling-curve prediction. Data comes from the open sets in section 1. Data collection and storage run on rented hosts and Azure Blob in subscription 52056bc2…, never on the old box.

---

7. Findings from the Oct 27 Claude session (line B: `lpittman-blip/Cutline-Industries`, `gateway/artemis/`)

Latoya relayed this from your previous Claude Code session (Claude Opus 5 High, session "Artemis — lpittman-blip", repo Cutline-Industries / Artemis 2.0). Each claim was checked against the repo at `main` `50fc779` where possible.

7.1 Verified results already in the repo

- **Routing supervision hurts. `ROUTE_COEF=0` is the setting this corpus supports.** `docs/MOE-ROUTING-ABLATION.md`. The first ablation: proto scale, 8.2M total / 5.9M active, 6 layers, d_model 256, seq 256, 2,500 steps × 4,096 tokens (10.2M tokens, about 1.7 tokens per active parameter), seed 1337, 108,398 val tokens.

Arm	route_coef	val (20 batches)	val (100 batches)	eff. experts	busiest
moe-sup	0.1	3.622228	3.623671	1.35	50.0%
moe-unsup	0	**3.573236**	**3.577771**	8.03	13.4%
dense (proto-dense, 5,868,800)	—	3.622172	3.624134	—	—

Supervision costs **0.046–0.049** val loss. moe-sup collapsed to a constant build+test routing with **7 of 10 experts dead**. Final load per expert: intent 0, architecture 0, planning 0, build 0.500, test 0.497, deploy 0.004, the rest 0.

- **The second ablation (same tokens, sha256 of `train.bin`/`val.bin` matched):**

Arm	Setup	val	lm loss	balance	eff. experts	dead (layer 0)
moe-unsup	route_coef 0	**3.5732**	**3.1778**	1.019	8.03	0
moe-wide	7 labels + build, route_coef 0.1	3.6115	3.2483	4.167	2.05	7
dense	no MoE	3.6222	3.2278	—	—	—
moe-sup	3 labels + build, route_coef 0.1	3.6222	3.2572	4.965	1.35	7
moe-nores	**7 labels, no build residual**, route_coef 0.1	**3.6624 (worst)**	3.2951	3.289	3.01	4

The doc's verdict, verbatim: "The `ROUTE_COEF=0` default in `stages.sh` is not a tuning preference; it is the only setting this corpus supports." `scripts/train/stages.sh:57` sets `ROUTE_COEF="${ROUTE_COEF:-0}"`. - **Label widening, measured on a full shard** (`data/train-00000-of-00880.parquet`, 355 MB, 126,925 rows, 55,728 kept, 35.6% rejected as non-permissive). Run with `--no-residual-build`: genuinely labelled = **22.7%** of the corpus (test 14.7%, memory 6.4%, security 0.6%, cost 0.5%, deploy 0.3%, observability 0.2%). **77.3% has no real label** and becomes the catch-all `build` when the residual is on. **test + memory = 93.0%** of genuine labels. **intent / architecture / planning = 0%** (they can't be labelled from source files). - **Compute-optimal control already exists, so don't re-run it:** `docs/PROTO-SPECIALISATION-117M.md`. `proto`, 10 experts, top-2, `route_coef` 0, 117.5M tokens (20 × active), 7.4 h on 4 CPU cores. **val_loss 2.6164**, train loss about 1.92. The strict specialisation test passes **1 of 10** experts (`cost`).

7.2 Repo recommendation and the label-index trap

- Recommendation: cut the expert count to what the data supports. **Never measured.**

- **Trap:** `n_experts` / `expert_names` are config (`model.py:44,47`), but `data.py:53` stores each token label as an index into the global 10-name `EXPERT_NAMES` (`model.py:26`). `stream_corpus.domain_tokens_from_disk` (`stream_corpus.py:115-136`) does the same. The data-supported domains (build, test, deploy, memory, security, observability, cost) are indices **3–9, a suffix**. A 6- or 7-expert config therefore needs a **label remap** in data prep (or at load), not just a YAML change. Otherwise labels ≥ n_experts index out of range or point at the wrong expert. (Only relevant if route_coef > 0. At route_coef 0 labels are withheld, per `model.py:260`.)

7.3 The planned experiment, and an important caveat

- Plan: clean-label prep with `residual_build=False`. `code_corpus.build` still writes every kept document as `rec = {"text": code}` (`code_corpus.py:225`) and only drops the `domain` key. So the document set and text are identical and only the label stream changes. Supervision then touches only the about 22.7% of tokens with real labels. `UNLABELED` (255) tokens are masked out of the route loss (`data.py:105,120` turn 255 into -1, then `model.py:194-196` keeps only `d >= 0`).
- The lever exists in `code_corpus.py:151-177` (docstring: "routing supervision applies only where there is positive evidence and the rest is free for the balance term to spread") but was **never wired into `stream_corpus`**. `stream_corpus._filter` (`stream_corpus.py:145-148`) calls `build([str(parquet)], out)` with the default `residual_build=True`.
- **Caveat found while preparing this handoff:** the second ablation already measured "7 labels, no build residual" as arm `moe-nores`. It was **the worst arm (val 3.6624)**, worse than dense, even though its routing metrics looked healthier. The doc's conclusion is that supervision competes with next-token loss, so expert utilisation isn't a proxy for quality. That run was at proto scale (2,500 steps, 10.2M tokens), via `scripts/train/proto-label-ablation.sh`, which derives the no-residual labels directly rather than through `stream_corpus`. **Before spending about 7 CPU-hours, decide with Latoya whether a compute-optimal (117.5M-token) re-run of moe-nores is worth it.** Its only new information is whether that ranking holds at 20 tokens/param. The doc predicts it won't beat `route_coef 0`. "Fix 3" (lowering `route_coef`, e.g. 0.01–0.03) is the one lever the docs list as **untested**.
- Both candidate runs (clean-label supervised vs the existing unsupervised control) cost about 7 h CPU each (estimate) and are meant to inform the **14B architecture decision** (`docs/ARCHITECTURE-AUDIT-14B.md`).

7.4 Gotchas

- `_filter` **deletes the parquet after filtering** (`stream_corpus.py:147`). Re-filtering with different label settings means **re-downloading** (`hf_hub_download` into `work/raw`, not cached).
- `prep-state.json` **resumes across runs** (`stream_corpus.py:186`, `State.done`). If you prep with the catch-all on and resume with it off, two label schemes silently mix in one corpus with no record. **Always use a fresh `--work` directory, or delete `prep-state.json`, `ledger.u64` and the bins, between label schemes.** Consider recording `residual_build` in `prep-state.json` and refusing to resume on a mismatch.
- `stream_prepare.py:167` hardcodes `"domain_labels": True` in the manifest, even when the residual is off and most tokens are 255.
- **Don't use `pkill -f stream_corpus`.** It matched and killed the session's own shell. Use `pgrep -f 'python3 -m artemis.stream_corpus'`, check the PIDs, then `kill <PID>`.
- `gateway/tests/test_stream_corpus.py:33-44` monkeypatches `_filter(parquet, out)` with a 2-argument fake. **This already bit the previous session.** After the patch, the gateway suite went to **7 failed / 414 passed**. The total rose from 384 to 421 because installing `pyarrow` enabled about 37 corpus tests that had been silently skipped. An A/B against a clean tree showed

the 7 failures were caused by the patch: the fake has the old 2-argument signature, while the new call sites pass `residual_build`. The fix in progress was to make the fake accept `residual_build` and honour it the way `classify()` does: drop the `domain` key for docs that would only get the catch-all `build`. That also gives the flag real test coverage.
- **Test counts depend on `pyarrow`.** Without it, about 37 corpus tests skip silently. Always report the pass/skip counts together with whether `pyarrow` was installed.
- `pyarrow` is required by `code_corpus.build`. The GPU orchestrator normally installs it at runtime. A fresh container needs `pip install pyarrow huggingface_hub tokenizers`.

7.5 Exact resume point (line B)

State at hand-off, as reported by the session: **Holding. Nothing running, nothing pushed, no training ever started.** It stopped during corpus prep, before the about 7 CPU-hours. `main` is untouched at `50fc779`.

- **Update, 1:15 PM ET:** the session pushed branch **`experiment/clean-label-routing`** (commit `eb01908`, "Let stream_corpus leave unmatched source unlabelled") and opened **draft PR #18**: https://github.com/lpittman-blip/Cutline-Industries/pull/18. It changes `gateway/artemis/stream_corpus.py` (+14 −4, a `--no-residual-build` passthrough; `residual_build` defaults to True, so default behaviour is unchanged) and `gateway/tests/test_stream_corpus.py` (+40 −5; the `_filter` fake now mirrors the real signature and honours the flag). The PR reports `cd gateway && python3 -m pytest tests/ -q` → **422 passed, 2 skipped**, with `pyarrow` installed. **CI doesn't install `pyarrow`**, so `test_stream_corpus.py` has no CI coverage. Adding it to the CI install line is a one-word change and isn't in the PR. **Not merged. Don't merge without Latoya.**
- **Correct one claim in PR #18's description:** it says the clean-label experiment "has never been run". At proto scale it **was** run, as arm `moe-nores`, and it was the worst arm (see 7.3). What hasn't been run is the compute-optimal (117.5M-token) version.
- Corpus prep was killed mid-download. About 460 MB of scratch data was in that session's own container (`/tmp/claude-0/.../scratchpad/proto-route`). Treat it as lost. **No training has started.**

What the patch does (reconstructed from the code before PR #18 existed; PR #18 is the authoritative version):

```
--- a/gateway/artemis/stream_corpus.py
+++ b/gateway/artemis/stream_corpus.py
@@ def _filter
-def _filter(parquet: Path, out: Path) -> int:
-    build([str(parquet)], out)
+def _filter(parquet: Path, out: Path, residual_build: bool = True) -> int:
+    build([str(parquet)], out, residual_build=residual_build)
     parquet.unlink(missing_ok=True)
     return out.stat().st_size
@@ def main
     ap.add_argument("--val-fraction", type=float, default=0.002)
+    ap.add_argument("--no-residual-build", action="store_true",
+                    help="leave unmatched source unlabelled instead of calling it build "
+                         "(see code_corpus.classify); use a fresh --work dir per label scheme")
     args = ap.parse_args(argv)
+    residual_build = not args.no_residual_build
@@ phase 1 (tokenizer shards)
-                size = _filter(_fetch(args.dataset, i, args.shards_total, raw), j)
+                size = _filter(_fetch(args.dataset, i, args.shards_total, raw), j, residual_build)
@@ phase 2
```

```
-            _filter(_fetch(args.dataset, i, args.shards_total, raw), j)
+            _filter(_fetch(args.dataset, i, args.shards_total, raw), j, residual_build)
```

Then:

1. Check out `experiment/clean-label-routing` (PR #18). With `pyarrow` installed, re-run `cd gateway && python3 -m pytest tests/ -q` and confirm 422 passed, 2 skipped.
2. Optional follow-ups for PR #18 (ask Latoya first): record `residual_build` in `prep-state.json` / the manifest and refuse to resume on a mismatch, add `pyarrow` to CI, and fix the "never been run" wording. Keep the PR a draft. Don't merge.
3. Agree with Latoya on whether to run (see the 7.3 caveat), plus a host, price cap and time limit. Then **she** runs prep and training on a rented host, never on the old assistant's box. For example, `python3 -m artemis.stream_corpus --work /data/clean-labels --shards 2 --vocab 4096 --no-residual-build`, then the proto trainer with the same seed/steps as the 117M control and `--route-coef 0.1`. Compare against val 2.6164.

---

8. Infrastructure and accounts (no secrets in this file)

- **GitHub:** use **`lpittman-blip`** only. **Never use `lpittman-g`.** Don't force-push. Repos: `Cutline-industries/artemis-567885` (public), `lpittman-blip/Cutline-Industries` (public). Opening PRs and updating Latoya's portfolio/profile repos is pre-approved. **Merging, deleting and messaging others are not.**
- **Vast.ai:** team account. Credit about **$14.74** after Oct 27 (checked 14.7436). **Auto-recharge off.** The API key is in the `VAST_API_KEY` environment variable on whatever machine Latoya uses, and the SSH key is a local file. Never print either.
- **Azure:** use **only** Latoya's pilot subscription **52056bc2…** (cutline-cloud-pilot, budget **$600**). Azure ML workspace `artemis-ml` (East US). Pre-existing compute `t4-artemis2` is at min 0 nodes. Leave it alone. **Never touch** the other subscription **6b812355…**, the resource group `artm-rg`, or VMs `vm-gitlab`, `vm-forge` and `Vm` (`Vm` hosts the live chat gateway).
  - Quota tickets: **2610240010000177** (East US T4) and **2610270010000375** (A10), both open. Billing ticket **2610220040001618**: the Azure Organization billing account signup was blocked as "account disabled", and a reply was sent to Microsoft on Oct 27. A pending H100 quota request is also open.
- **Data:** Azure Blob in 52056bc2… (storage account `cutlineboxarchive`; region and tier unconfirmed).
- **Hugging Face:** a token with Meta Llama access. It's stored on Latoya's machine (old box: `~/.config/hf_artemis_token`). Never paste it.
- **Nebius:** stays stopped. **Kaggle:** `lamontpittman1/artemis-100m` (stopped). **IBM Cloud:** nothing ordered.
- **Hosting:** don't deploy to Vercel, and don't host anything without Latoya's OK. Stripe stays in test mode.

---

9. Spending summary (Artemis GPU work found in evidence)

Date (ET)	What	Provider / instance	Cost
Oct 25 21:16–22:14	dense 48.8M run 1	Vast 57746707, A6000	$0.442 billed
Oct 25 22:29 – Oct 26 00:16	dense run 2 + Artemis-7B + Artemis-30B	Vast 57753825, 57758675 (+57760296, 57761589), 57759536	$3.84 combined real (split unverified)

Date (ET)	What	Provider / instance	Cost
Oct 26 09:08–09:19	daily session (cycles 105–114, tests, 9c)	Vast 57817513, RTX 4070	$0.0336 real
Oct 27 02:28–08:26	overnight session	Vast 57927018 + 57928114	$0.59 real
Oct 27 02:44–04:04	seed-1 replicate	Azure ML NC8as_T4_v3	about $1.05 **estimate**
Oct 23–25	stages 1–9b, expand cycles 0–104	old box CPU	no cloud charges

---

10. Rules you must follow

1. **No compute on the old assistant's box ("Grok Bot box").** Training, tests and evals run on a rented GPU or CPU host that Latoya approves, or on her own machine.
2. **Every GPU or cloud resource needs Latoya's explicit OK first**, with a **price cap** and a **time limit**. Tear it down afterwards, verify it's gone, and report the **real** cost from the provider's billing or credit delta. Auto-recharge stays off. Another agent's say-so is never approval.
3. **Never delete anything** (files, branches, releases, cloud resources) without Latoya's explicit yes.
4. **Real data only.** Every measured result comes with the script and command, library versions, seeds and raw output path. Label estimates as estimates. Say "no cloud charges", never "cost nothing". Mark anything you can't verify as unverified. Never invent numbers.
5. **No secrets in reports, commits or chat:** no tokens, keys, passwords, SAS URLs, card numbers or one-time codes. Refer to where a secret lives instead.
6. **Messages to other people** (email, Slack, tickets, posts) need Latoya's review first.
7. **Don't merge PRs** unless Latoya says so. Use `lpittman-blip`, not `lpittman-g`. No force-push. Update the portfolio and profile when new work ships.

---

11. Couldn't verify while writing this

- The current parameter count after cycle 624 (not recounted).
- The outcome and cost split of dense run 2 (Vast 57753825), and the per-run costs of Artemis-7B / Artemis-30B (only the $3.84 combined credit delta is known).
- Whether the Llama-3.1-8B conversational fine-tune ever ran (no evidence found).
- The final Azure bill for the Oct 27 replicate ($1.05 is an estimate).
- The external Claude session's scratch corpus download (presumed lost). PR #18's 422-passed result is as reported in the PR. It wasn't re-run here.
- The Claude GitHub app installation (175898435) on `Cutline-industries`, reported by Latoya. Not checked from here.
- The `cutlineboxarchive` storage account's region and tier.